

Last revised : 24.10.2004
Total number of pages: 26

USER MANUAL FOR

MATRIX FITTER

version 1.0 for Matlab

Bjørn Gustavsen
SINTEF Energy Research
N-7465 Trondheim
NORWAY

E-mail: bjorn.gustavsen@energy.sintef.no
Fax: +47 73597250

1. INTRODUCTION	2
2. THEORY	3
3. MODIFICATIONS OF CODE.....	4
4. PROGRAM FILES	5
5. ROUTINE DESCRIPTION	7
5.1 Purpose	7
5.2 Function call	7
5.3 User tips	9
6. TUTORIAL	10
6.1 Running the example.....	10
6.2 Calculated results	
6.2.1 Frequency domain identification.....	10
6.2.2 Passivity enforcement	14
6.2.3 Generation of equivalent electrical network.....	17
6.2.4 Generation of realizations	
6.3 Time domain calculation using ATP	18
7. REFERENCES	20
IEEE paper describing the program package	21

1. INTRODUCTION

This manual describes a set of routines for system identification of multiterminal systems in the frequency domain. The output is a realization of the type used in state equations and an equivalent electrical network which can be directly imported into ATP-EMTP for simulation of electromagnetic transients. The routines are written in the Matlab language and do not require usage of toolboxes.

The identification is based on rational approximation by the Vector Fitting technique (VF) [1], which is very robust and accurate. Since 1998 a routine has been available in the public domain (`vectfit.m`) which has the capability of fitting scalar functions and vector functions using only stable poles. The objective of this new collection of files is to assist the user in using the VF-technique for system identification of multiterminal systems. Emphasis has therefore been put on clarity and simplicity, rather than on smart programming tricks.

The applied strategy is to stack all matrix columns of $H(s)$ into a single column which is next fitted using a sparse version of VF (`spavectfit.m`). This results in a realization with repeated poles. The resulting realization leans itself well to passivity enforcement by the QP-approach [2], which is important for ensuring a stable time domain simulation. (The QP-approach is not included in this package; instead the simplified approach described in [2] is supplied). An experienced user can easily adopt the program system to other fitting strategies, e.g. columnwise fitting, which requires less computer resources.

An overview of the program system and its usage has been published in [3], which is appended to this document. Some changes have been made as compared to this reference, in order to improve on clarity. The modifications are described in section 3.

The program system has been tested on Matlab version 5.3 and 6.0.

Download site:

<http://www.energy.sintef.no/Produkt/VECTFIT/index.asp>

Note 1: The program system is in the public domain and can be freely used by anyone. It is only required that if the program code or parts of it is used in a scientific work, or in a commercial program, then reference should be made to [1] and [3]. If sending the code to a third party, it is required that the entire package of files is shipped as a single entity.

Note 2: The QP-approach is available at the download site in routine `QPpassive.m`.

2. THEORY

Consider a column vector $h(s)$ of length n . VF approximates $h(s)$ with a sum of partial fractions

$$h(s) = \sum_{m=1}^N r_m \cdot \frac{1}{s - a_m} + d + se \quad (1.1)$$

where r_m is a vector of length n and N is the order of the approximation. Terms d and e are optional. All poles are stable.

Assume that all columns of a matrix H of size n by n have been placed in a single column and fitted using VF. We then get a column h in (1.1) of length n^2 . By partitioning the obtained r, d, e into vectors of length n and stacking these side-by-side we can obtain a realization as shown in (1.2), with H, R, D, E being of size n by n .

$$H(s) = \sum_{m=1}^N R_m \cdot \frac{1}{s - a_m} + D + sE \quad (1.2)$$

The realization (1.2) can be directly converted into the form

$$H(s) = C(sI - A)^{-1}B + D + sE \quad (1.3)$$

This is achieved by building the matrices in (1.3) from those in (1.2) as shown in fig. 1 ($n=2$, $N=3$). It is seen that C is established by copying the columns of the R -matrices into the appropriate locations in C . B has columns of ones and A is diagonal with the poles repeated n times.

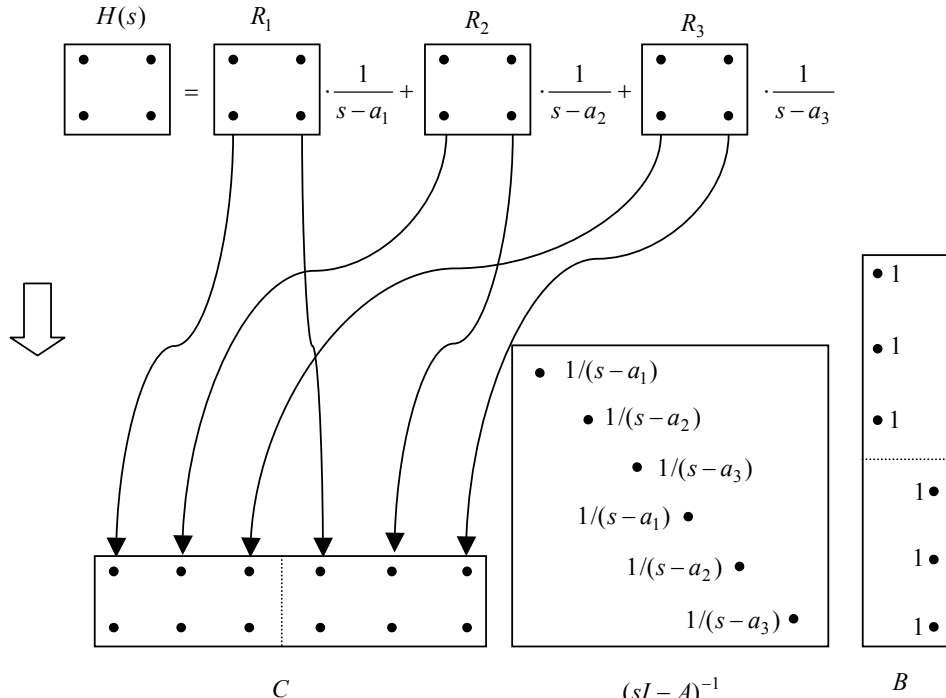


Fig. 1. Converting (1) into a state-space model $(sI - A)^{-1}$

Note that the program system produces both the realizations (1.2) and (1.3). The matrices A, B, C, D defines a state space model of the system (when $E=0$).

The realization in (1.2) can be converted into an equivalent electrical network when $H(s)$ is an admittance matrix. In this network the terminals represent the nodes. Parallel branches are then calculated and placed between all nodes, and between all nodes and ground. (Branches representing complex poles have an internal node). This is shown in fig. 2 when $n=3$. Each branch in fig. 2 consists of as many parallel sub-branches as the order of the approximation. In addition, one branch results from D and one from E . The calculation of branches is shown in detail in [3]

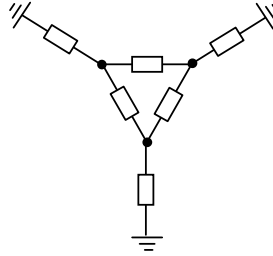


Fig. 2 Equivalent electrical network for system with 3 terminals

3. MODIFICATIONS OF CODE

Some changes have been made to the code as compared to the description in [3].

- The passivity enforcement is done by the Simplistic Approach described in [2]. In the provided computer code, the separate enforcement for D and E has been removed. This was done in order to simplify the code.
- The routine `spavectit.m` has been modified to allow individual frequency dependent weighting of all matrix elements.

4. PROGRAM FILES

The package consists of the following 8 files:

user_manual.pdf	% This document
tutorial.m	% m-file for tutorial example shown in this manual (see Section 6)
mainfit.m	%Main routine
mtrxvectfit.m	%Converting $H(s)$ into vector $h(s)$, calling spavectfit.m
spavectfit.m	%Approximating $h(s)$ with rational functions
passive.m	%Passivity enforcement
netgen.m	%Generation of electrical network for ATP time domain simulation
circuit.atp	%ATP data file for example in Section 6.

The program system is started by calling routine `mainfit.m`. However, the individual routines can also be used independently.

An overview of the calculation procedure is shown in fig. 1.

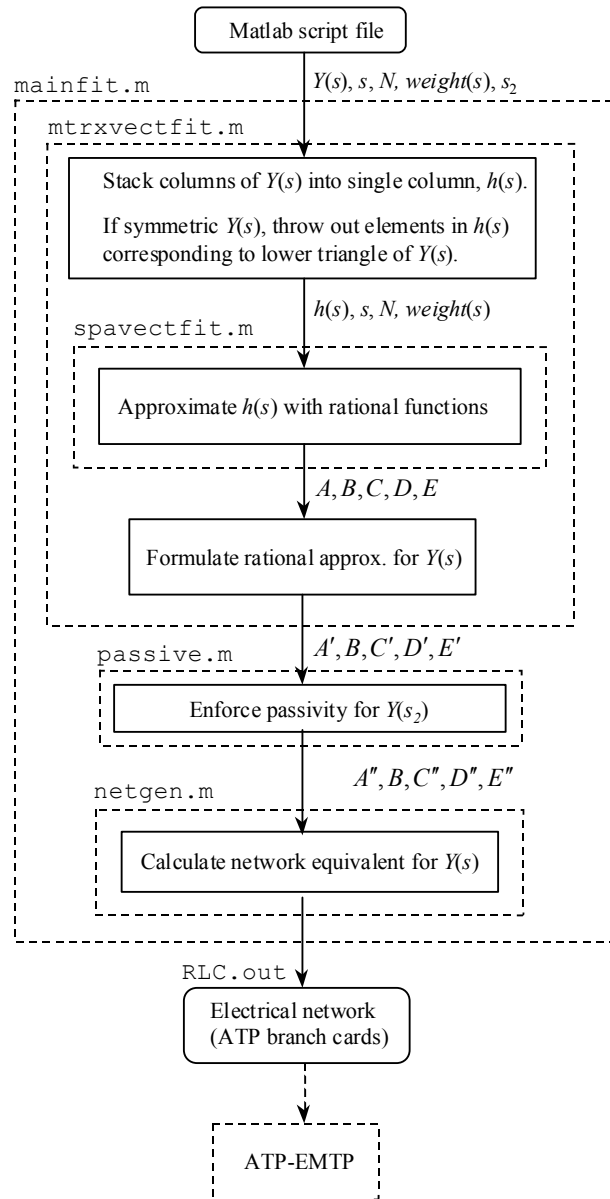


Fig. 1 Overview of calculation procedure

5. ROUTINE DESCRIPTION

5.1 Purpose

`mainfit.m` approximates the frequency response $H(s)$ with rational functions in the form of a sum of partial fractions where each element i,j is expressed as

$$H_{ij}(s) = \sum_{m=1}^N \frac{SERC_{ijm}}{s - SERA_m} + SERD_{ij} + s \cdot SERE_{ij} \quad (5.1)$$

where $SERD$ and $SERE$ are optional. This is achieved by stacking the columns of $H(s)$ into a single column (`mtrxvectfit.m`) which is subjected to fitting by a sparse version of Vector Fitting (`spavectfit.m`).

In situations where $H(s)$ represents an admittance matrix, the user has the option to enforce passivity for the rational approximation, i.e.

$$eig\{\text{Re}(H(s))\} > 0 \quad (5.2)$$

This is achieved by a small modification to $SERD$ which is calculated using routine `passive.m`, which applies the "Simplistic Approach" described in [2].

In addition, `mainfit.m` converts the expression in (5.1) into the realization

$$H(s) \approx C(sI - A)^{-1}B + D + sE \quad (5.3)$$

where A is diagonal. (Again, D and E are optional.)

Finally, the rational approximation is converted into an equivalent electrical network which is written to file (`RLC.out`). The file can be imported into an ATP data file using the `$INCLUDE` feature of ATP. The calculation of the network is explained in [3]. (Note: the content of `RLC.out` is only meaningful when $H(s)$ is an admittance matrix!).

The node names in the ATP-circuit are

A___1, A___2, A___3, etc

5.2 Function call

The program system is invoked by calling the driver routine `mainfit.m`, which calls the other routines as needed.

The listing below defines the function call for `mainfit.m` and the required input and output variables. The routine is seen to return the rms-error of the approximation as well as realizations in the form of (5.1) and (5.3). In addition, branch cards for ATP are written to the file `RLC.out`. The input consists of the frequency responses $H(s)$ (contained in `bigY`), the frequency samples (s), and a number of control parameters.

It is assumed that $H(s)$ is complex conjugate with respect to s . Only positive frequency samples should be specified.

```
function [SERA,SERC,SERD,SERE,A,B,C,D,E,rmserr]=...
    mainfit(bigY,symmflag,s,N,SERAstart,startpoleflag,weight,...
        kill,asymptflag,spy,logx,logy,errplot,skip,Niter,passiveflag,s_pass);

% =====
% = Routine: mainfit.m =
% = Version 1.0 =
% = Last revised: 19.03.2002 =
% = Programmed by: Bjorn Gustavsen, =
% = SINTEF Energy Research, N-7465 Trondheim, NORWAY =
% = This file is part of the "matrixfitter-package" =
% =====
% PURPOSE : Calculate the rational least squares approximation of a matrix Y(s)
%
% - as a sum of partial fractions:
%       N   SERCijm
%       Yij(s)=SUM(----- ) +SERDij +s*SEREij ("ij" denotes element i,j)
%       m=1 (s-SERAm)
%
% - as a state-space model:
%       Y(s)=C*(s*I-A)^(-1)*B +D +s*E
%
% - as an equivalent electrical network in the form of branch cards for ATP.
%   (only applies when Y(s) is an admittance matrix)
%
%       with option of passivity enforcement
%
% INPUT :
%
% bigY(Nc,Nc,Ns) : matrix function to be fitted.
%                  Nc : dimension of Y(s)
%                  Ns : number of frequency samples
%
% symm_flag=0 --> Y(s) is treated as unsymmetric
% symm_flag=1 --> Y(s) is treated as symmetric (increases efficiency and saves
%                                                memory)
%
% s(Ns,1) : vector of frequency points [rad/sec]
%
% N : number of starting poles (used when startpoleflag~=0)
%
% SERAstart(N,1) : vector of starting poles [rad/sec]
%
% startpoleflag=0 --> SERAstart is used as starting poles
% startpoleflag=1 --> automatic gen. of complex, linearly spaced starting poles
% startpoleflag=2 --> automatic gen. of complex, log. spaced starting poles
% startpoleflag=3 --> automatic gen. of real, log. spaced starting poles
%
% weight(Nc,Nc,Ns): the rows in the system matrix of the least squares problem
%                   are weighted using this array.
%                   The elements in weight correspond directly to the elements in
%                   bigY. Thus, each element in bigY can be weighted individually.
%                   If no weighting is desired, use:
%                   weight=ones(Nc,Nc,Ns) --> Same weight for all samples
%
% kill=0 --> unstable poles are kept unchanged
% kill=1 --> unstable poles are deleted
% kill=2 --> unstable poles are 'flipped' into the left half plane
%           (kill=2 is the recommended choice)
%
% asymptflag=1 --> SERD=0, SERE=0, D=0, E=0
% asymptflag=2 --> SERD~=0, SERE=0, D~=0, E=0
% asymptflag=3 --> SERD~=0, SERE~=0, D~=0, E~=0
%
% spy=1 --> Plotting of results on screen:
%           figure(1): magnitude functions
%           figure(2): phase angles
%           cyan trace : Y
%           magenta trace : (Y)fit
%           green trace : |(Y - (Y)fit)|
%
% logx=0 --> Plotting using linear abscissa axis
% logx=1 --> Plotting using logarithmic abscissa axis
%
% logy=0 --> Plotting using linear ordinate axis
% logy=1 --> Plotting using logarithmic ordinate axis
% errplot=1--> Will include fitting error in the plot
%
% skip=1 --> Will not plot results during iterations (pole identification)
```



```
%
% Niter : N.o. iterations used for identifying poles
%         (recommended value: 3 or higher)
%
% passiveflag=0 --> Passivity will not be enforced
% passiveflag=1 --> Passivity is enforced
%
% s_pass(Ns,1) : Vector of frequency samples for which passivity is enforced
%
%
% OUTPUT :
%
%         N
% Yij(s)=SUM( SERC(i,j,m)/(SERA(m) ) +Dij +s*Eij  (i=1,Nc,j=1,Nc)
%           m=1
%
% Y(s)=C*(s*I-diag(A)) ^(-1)*B +D +s*E
%
% SERA(N,1)
% SERC(Nc,Nc,N)
% SERD(Nc,Nc) : matrix D (is produced if asympflag=2 or 3)
% SERE(Nc,Nc) : matrix E (is produced if asympflag=3)
%
% A(N*N,1) : matrix A (diagonal)
% B(Nc*N,Nc) : matrix B
% C(Nc,Nc*N) : matrix C
% D(Nc,Nc) : matrix D (is produced if asympflag=2 or 3)
% E(Nc,Nc) : matrix E (is produced if asympflag=3)
%
% rmserr : root-mean-square error of approximation for Y(s)
```

5.3 User tips

All variables shown in the function call (section 5.2) have to be included. In situations where a variable is not used, it is permissible to place a "[]" for the variable :

- When automatic generation of starting poles is selected (startpoleflag=1, 2 or 3), SERAstart can be given as [].
- With startpoleflag=0, SERAstart has to be specified. N can then be given as [] (because N is calculated internally as length(SERAstart)).
- When passivity is not to be enforced (passiveflag=0), s_pass can be replaced with [].

A set of starting poles is required by routine spavectfit.m (which is called via mainfit.m and mtrxvectfit.m, see fig. 1). In general, the starting poles should be as close as possible to the final poles, in order to obtain a faster convergence. For situations where the frequency response has many resonance peaks, it is recommended to use startpoleflag=1 or 2. (Results in complex pairs with weak attenuation that are linearly distributed (=1) or logarithmically distributed (=2) over the frequency range defined by s).

- For situations where the frequency response is very smooth, one can also use startpoleflag=3. (Results in real poles that are logarithmically distributed over the frequency range defined by s).
- Advanced users may choose to specify SERAstart directly (startpoleflag=0).

If one wishes to obtain a state-space model then the term sE in (5.3) should be enforced to be 0. This is achieved by choosing asympflag=1 or 2 in the function call (see Section 5.2)

The number of iterations used by Vector Fitting (Niter) should normally be 3 or higher.

In some situations the matrix $H(s)$ may contain very small elements. If one wishes to achieve a high relative accuracy for all elements (including the small ones) then one should utilize the weight array in the function call (see Section 5.2), e.g.

```
for row=1:Nc
    for col=1:Nc
        weight(row,col,:)=1./ (norm(abs(squeeze(bigY(row,col,:)))));
    end
end
```

6. TUTORIAL

Consider the electrical network in fig. 2 (quantities given in units $[\Omega]$, $[H]$, $[F]$). We wish to calculate a black box model of the equivalent with respect to terminals 1 and 2 when only the frequency response at the terminals are known.

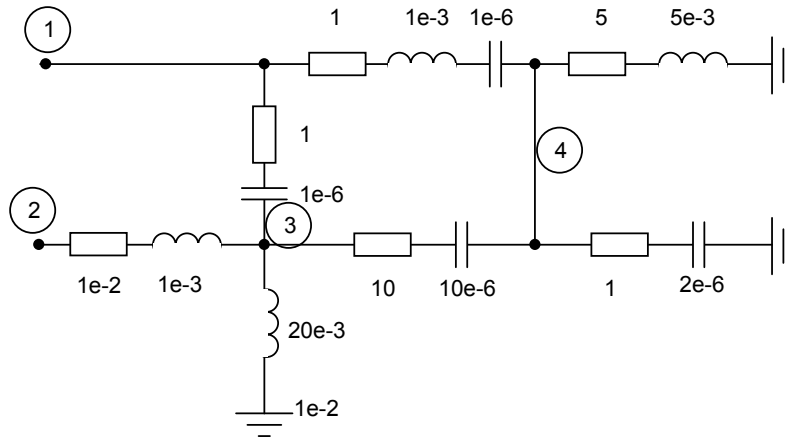


Fig. 2 Electric circuit

6.1 Running the example

Running `tutorial.m` from Matlab's command window gives the output to screen shown below.

```
>> tutorial
Stacking matrix columns into single column...
Vector Fitting...
Enforcing passivity...
Generating equivalent electrical network...
Generating statespace model...
rms-error : 5.697826e-015
>>
```

The rms-error of the resulting state-space-model is seen to be $5.69e-15$.

6.2 Calculated results

6.2.1 Frequency domain identification

In `tutorial.m`, an admittance matrix (4 by 4) is first established for the circuit in fig. 2 and is then reduced with respect to terminals 1 and 2, giving a 2 by 2 admittance matrix `bigY`. This is achieved using the code section shown below. Fig. 3 shows the elements of `bigY` (magnitude functions).

```
% tutorial.m
%
% Example described in user_manual.pdf
%
% =====
% = Routine: tutorial.m =
% = Version 1.0 =
% = Last revised: 19.03.2002 =
% = Programmed by: Bjorn Gustavsen, =
% = SINTEF Energy Research, N-7465 Trondheim, NORWAY =
% = This file is part of the "matrixfitter-package" =
% =====
%
clear all

Ns=501; %Number of frequency samples
Nc=2; %Size of Y (after reduction)
bigY=zeros(Nc,Nc,Ns);
Y=zeros(4,4);
s=2*pi*i*logspace(1,5,Ns);

%Component values:
R1=1; L1=1e-3; C1=1e-6;
R2=5; L2=5e-3;
R3=1; C3=1e-6;
L4=1e-3;
R4=1e-2; L5=20e-3;
R6=10; C6=10e-6;
R7=1; C7=2e-6;

%Building Y, reduction:
for k=1:Ns
    sk=s(k);
    y1=1/( R1+sk*L1+1/(sk*C1) );
    y2=1/( R2+sk*L2 );
    y3=1/( R3+1/(sk*C3) );
    y4=1/( R4+sk*L4 );
    y5=1/(sk*L5);
    y6=1/( R6+1/(sk*C6) );
    y7=1/( R7+1/(sk*C7) );

    Y(1,1)= y1+y3;
    Y(2,2)= y4;
    Y(3,3)= y3 +y4 +y5 +y6;
    Y(4,4)= y1 +y2 +y6 +y7;

    Y(1,3)=-y3; Y(1,4)=-y1;
    Y(2,3)=-y4;
    Y(3,1)=-y3; Y(3,2)=-y4; Y(3,4)=-y6;
    Y(4,1)=-y1; Y(4,3)=-y6;

    %Eliminating nodes 3 and 4:
    Yred=Y(1:2,1:2)-Y(1:2,3:4)*Y(3:4,3:4)^(-1)*Y(3:4,1:2);
    bigY(:,:,k)=Yred;

    %bigY(1,1,k)=bigY(1,1,k)-1e-4; %Modifying element (1,1) to see the
    %effect of pass. enf.
end
```

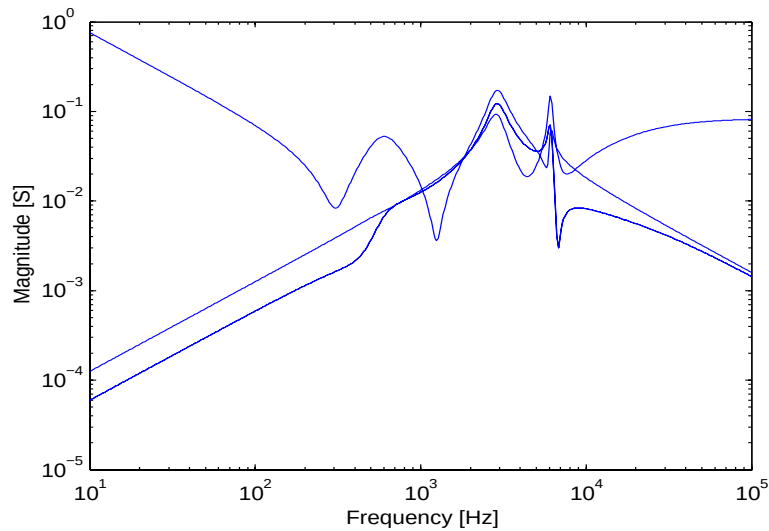


Fig. 3 Elements of Y(s)

The frequency domain responses in bigY are next fitted by a call to mainfit.m :

```
%=====
%          MATRIX FITTING          =
%=====
N=8; %Order of approximation
symmflag=0; %Treat Y as a symmetric matrix (fit upper triangle only)
SERAstarr=[]; %Specify only if startpoleflag==0
startpoleflag=2; %Will use logarithmically spaced, complex poles
weight=ones(2,2,Ns); %Will use same weight for all elements, at all frequencies
kill=2; %Enforce stable poles
asymptflag=3; %Fitting will include terms D and E
spy=1; %Will plot the fitted matrices (figure(1), figure(2))
logx=1; %Plotting is done using logarithmic abscissa axis
logy=1; %Plotting is done using logarithmic ordinate axis
errplot=1; %Will include fitting error in the plot
skip=0;
Niter=2; %Number of iterations by Vector Fitting
passiveflag=1; %Will enforce passivity for the fitted Y
s_pass=2*pi*i*logspace(0,6,301); %Frequency samples where passivity is enforced

[SERA,SERC,SERD,SERE,A,B,C,D,E,rmserr]=...
    mainfit(bigY,symmflag,s,N,SERAstarr,startpoleflag,weight,...
        kill,asymptflag,spy,logx,logy,errplot,skip,Niter,passiveflag,s_pass);
```

mainfit.m calls mtrxvectfit.m which in turn calls spavectfit.m. The plots in fig. 4 are produced by spavectfit.m. It is seen that the approximation is highly accurate.

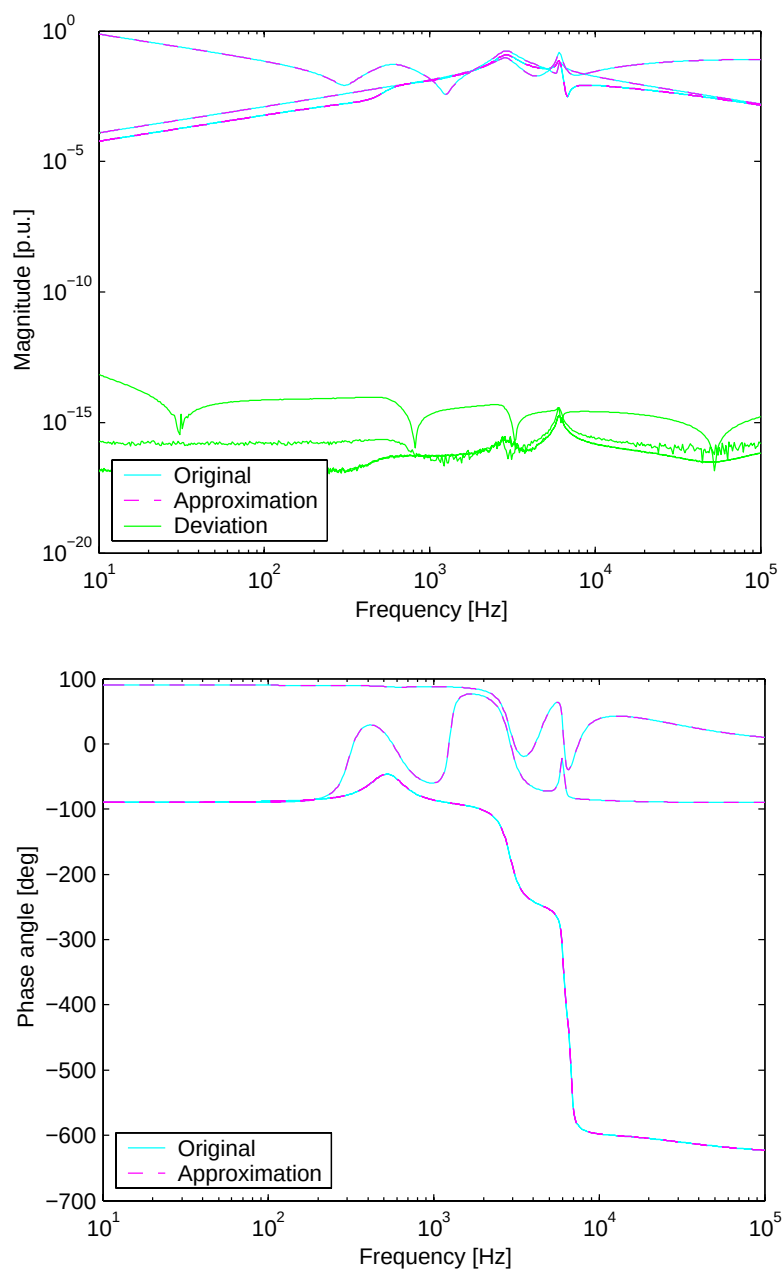


Fig. 4 Generated plots

6.2.2 Passivity enforcement

The code next calls `passivity.m` which enforces passivity for the obtained state-space model by requiring the eigenvalues of $\text{eig}(\text{real}(Y))$ to be positive. The following plot (figure(3)) is produced:

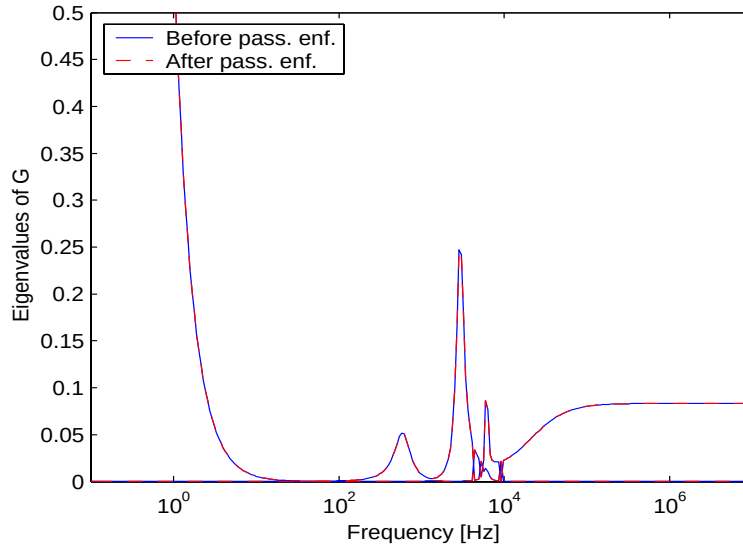


Fig. 5 Effect of passivity enforcement

In this case all eigenvalues were stable in the first place so that the enforcement had no effect.

To see the effect of the enforcement, `bigY` was modified by subtracting a value of $1e-4$ from element (1,1) by removing the comment `%"` on line 56 in `tutorial.m`. Fig. 6 shows that the enforcement now mitigates a negative eigenvalue. (The vertical lines represent artificial eigenvalue switchovers and should be ignored when interpreting the result)

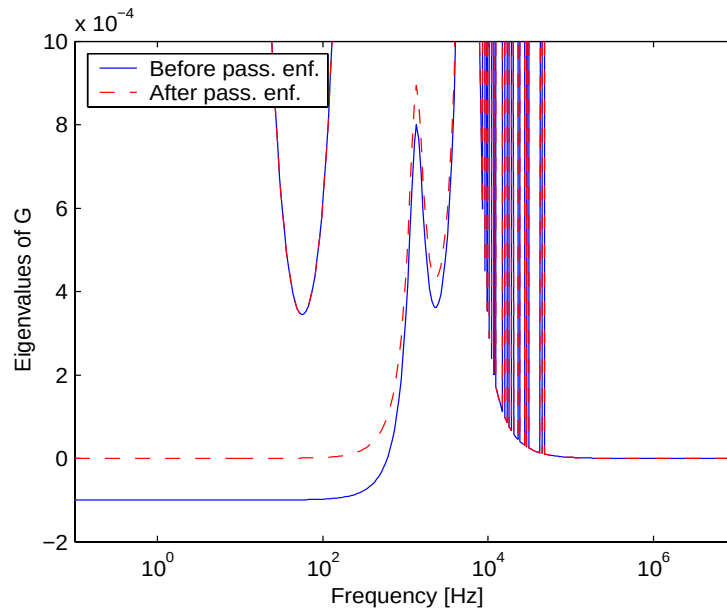


Fig. 6 Effect of passivity enforcement on modified example

6.2.3 Generation of equivalent electrical network

mainfit.m next calls netgen.m which produces a file RLC.out with contents as follows:

```
$VINTAGE,1
C <BUS1><BUS2><BUS3><BUS4>< OHM      ><   milliH   ><   microF   >
C
C (1,1)
A ____1      1.200000e+001
A ____1      -2.205051e+006  -4.630607e+009
A ____1      -1.219595e+001  -9.471947e-002
A ____1A 3__1      3.766458e+002  1.263769e+002
A 3__1      -2.270289e+003
A 3__1      4.713746e-001
A ____1A 5__1      -2.573387e+001  -5.835992e+000
A 5__1      -1.145657e+004
A 5__1      -5.191300e-001
A ____1A 7__1      1.094959e+001  2.397454e+000
A 7__1      -1.393562e+003
A 7__1      2.820713e-001
C (1,2)
A ____1A ____2      2.205051e+006  4.630607e+009
A ____1A ____2      2.391753e+002  1.857548e+000
A ____1A 3__12      -3.856542e+002  -1.338030e+002
A 3__12A ____2      2.625697e+003
A 3__12A ____2      -4.553681e-001
A ____1A 5__12      1.376913e+001  1.861842e+000
A 5__12A ____2      -2.323535e+002
A 5__12A ____2      1.527367e+000
A ____1A 7__12      -9.973998e+000  -6.609108e+000
A 7__12A ____2      -1.885228e+004
A 7__12A ____2      -1.031863e-001
C (2,2)
A ____2      9.999999e-003  2.100000e+001
A ____2      -2.269793e+002  -1.762828e+000
A ____2A 3__2      1.422631e+001  8.372073e+000
A 3__2      3.239494e+002
A 3__2      8.905312e+000
A ____2A 5__2      1.266286e+001  4.041532e+000
A 5__2      9.129339e+002
A 5__2      7.583205e-001
A ____2A 7__2      -8.845663e-001  4.235334e+000
A 7__2      2.784761e+003
A 7__2      1.608827e-001
$VINTAGE,0
```

Nodes A____1 and A____2 represent the terminals of the equivalent.

This circuit is conveniently imported into ATP-EMTP using the \$INCLUDE statement imbedded in an ATP data file, see section 6.3.

6.2.4 Generation of realizations

Finally, realizations of the form (5.1) and (5.3) are returned in the function call to `mainfit.m`
(SERA, SERB, SERC, SERD, SERE, A, B, C, D, E,) .

Diagonal elements of A in (5.3):

```
A =  
-4.7619e-001  
-1.2876e+005  
-1.0229e+003 +3.5994e+003i  
-1.0229e+003 -3.5994e+003i  
-2.2888e+003 +1.8044e+004i  
-2.2888e+003 -1.8044e+004i  
-1.0116e+003 +3.8290e+004i  
-1.0116e+003 -3.8290e+004i  
-4.7619e-001  
-1.2876e+005  
-1.0229e+003 +3.5994e+003i  
-1.0229e+003 -3.5994e+003i  
-2.2888e+003 +1.8044e+004i  
-2.2888e+003 -1.8044e+004i  
-1.0116e+003 +3.8290e+004i  
-1.0116e+003 -3.8290e+004i
```

(all poles are seen to be stable!)

```
B =  
1      0  
1      0  
1      0  
1      0  
1      0  
1      0  
1      0  
1      0  
1      0  
0      1  
0      1  
0      1  
0      1  
0      1  
0      1  
0      1  
0      1  
0      1  
0      1
```


C.'

ans =

```
-3.5085e-015          -2.1595e-007
-1.0019e+004          -5.3834e+002
 2.1958e-001 +2.2124e-001i  3.7368e+000 +1.9303e+000i
 2.1958e-001 -2.2124e-001i  3.7368e+000 -1.9303e+000i
 1.8288e+002 +6.5934e+001i -2.6855e+002 -7.6004e+001i
 1.8288e+002 -6.5934e+001i -2.6855e+002 +7.6004e+001i
 1.3290e+002 +1.8383e+001i  7.5653e+001 +9.8304e-001i
 1.3290e+002 -1.8383e+001i  7.5653e+001 -9.8304e-001i
-2.1595e-007          4.7619e+001
-5.3834e+002          -2.8926e+001
 3.7368e+000 +1.9303e+000i  5.5986e+001 +9.2914e+000i
 3.7368e+000 -1.9303e+000i  5.5986e+001 -9.2914e+000i
-2.6855e+002 -7.6004e+001i  3.9227e+002 +8.1793e+001i
-2.6855e+002 +7.6004e+001i  3.9227e+002 -8.1793e+001i
 7.5653e+001 +9.8304e-001i  4.2401e+001 -4.7459e+000i
 7.5653e+001 -9.8304e-001i  4.2401e+001 +4.7459e+000i
```

D =

```
8.3333e-002 -5.0441e-017
-5.0014e-017 -5.0855e-016
```

E =

```
1.8488e-022  1.2815e-022
1.2973e-022  3.6974e-021
```

6.3 Time domain calculation using ATP

Consider the situation that a unit step voltage is applied to terminal 1 and we wish to calculate the resulting current flowing into terminal 2, see fig. 7.

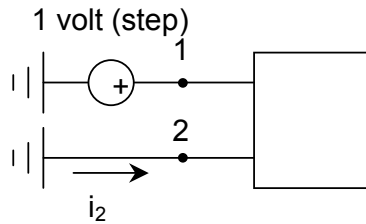


Fig. 7 ATP-simulation

An ATP-file for the simulation case in fig. 7 is shown below. (A $1\ \mu\Omega$ resistor is used for the current measurement). Note that the equivalent electrical network is imported using the \$INCLUDE statement, and that the node names are A___1 and A___2.

```
BEGIN NEW DATA CASE
C
C   Example described in user_manual.pdf
C
C   =====
C   =   File:   circuit.atp           =
C   =   Version 1.0                   =
C   =   Last revised: 19.03.2002      =
C   =   Programmed by: Bjorn Gustavsen, =
C   =   SINTEF Energy Research, N-7465 Trondheim, NORWAY =
C   =   This file is part of the "matrixfitter-package" =
C   =====
C
C $DUMMY, XYZ000
C
C FIRST MISCELLANEOUS DATA CARD
C $DUMMY, XYZ000
C FIRST MISCELLANEOUS DATA CARD
C dT >< Tmax >< Xopt >< Copt >
C 1.000E-6 .005
C
C SECOND MISCELLANEOUS DATA CARD
C 1-8 9-16 17-24 25-32 33-40 41-48 49-56 57-64 65-72 73-80
C PRINT PLOT NETWORK PR.SS PR.MAX I PUN PUNCH DUMP MULT. DIAGNOS
C 0=EACH 0=EACH 0= NO 0= NO 0= NO 0= NO 0= NO INTO ENERG. PRINT
C K=K-TH K=K-TH 1=YES 1=YES 1=YES 1=YES 1=YES DISK STUDIES 0=NO
C 10000 1 0 0 1
C
C BRANCHES
C 3456789012345678901234567890123456789012345678901234567890
C 3-8 9-14 15-20 21-26 27-32 33-38 39-44
C NODE NAMES REFERENCE RES. IND. CAP. (OUTPUT IN COLUMN 80)
C BRANCH MH UF I= 1
C <BUS1><BUS2><BUS3><BUS4> OHM OHM UMHO V= 2
C I.V 3
C A___2 1.E-6 1
$INCLUDE d:\user\matrixfit2001\manual\rlc.out
C
C BLANK CARD TERMINATING BRANCH CARDS
C SWITCH CARDS
C 3456789012345678901234567890123456789012345678901234567890
C 3-8 9-14 15-24 25-34 35-44 45-54 55-64 65-74
C (OUTPUT OPTION IN COLUMN 80)
C NODE NAMES IE FLASHOVER SPECIAL REFERENCE
C TIME TO TIME TO OR VOLTAGE REQUEST SWITCH-NAME
C BUS1 BUS2 CLOSE OPEN NSTEP WORD BUS5 BUS6
C
C BLANK CARD TERMINATING SWITCH CARDS
C SOURCE CARDS
C 3456789012345678901234567890123456789012345678901234567890
C COLUMN 1.2: TYPE OF SOURCE 1 17.(E.G. 11-13 ARE RAMP FUNCTIONS. 14 = COSINE)
```

```

C COLUMN 9.10: 0=VOLTAGE SOURCE. 1=CURRENT SOURCE
C 3-8 11-20 21-30 31-40 41-50 51-60 61-70 71-80
C NODE AMPLITUDE FREQUENCY TO IN SEC AMPL-A1 TIME-T1 T-START T-STOP
C NAME IN HZ DEGR SECONDS SECONDS SECONDS
14A 1 1.0 0.001 0.
C
BLANK CARD TERMINATING SOURCE CARD
C NODE VOLTAGE OUTPUT
C 34567890123456789012345678901234567890123456789012345678901234567890
C 3-8 9-14 15-20 21-26 27-32 33-38 39-44 45-50 51-56 57-62 63-68 69-74 75-80
C BUS1 BUS2 BUS3 BUS4 BUS5 BUS6 BUS7 BUS8 BUS9 BUS10 BUS11 BUS12 BUS13
C A 1A 2
C NBY1A NBY1B NBY1C NBY6A NBY6B NBY6C NEUT NBY6G GENA GENB GENC
BLANK CARD TERMINATING OUTPUT CARDS
BLANK CARD ENDING PLOT CARDS
BEGIN NEW DATA CASE

```

Fig. 8 shows the current calculated by ATP. In the same figure is also shown the resulting current as calculated using a detailed representation of the network. The equivalent electrical network is seen to produce the same waveform as the original network.

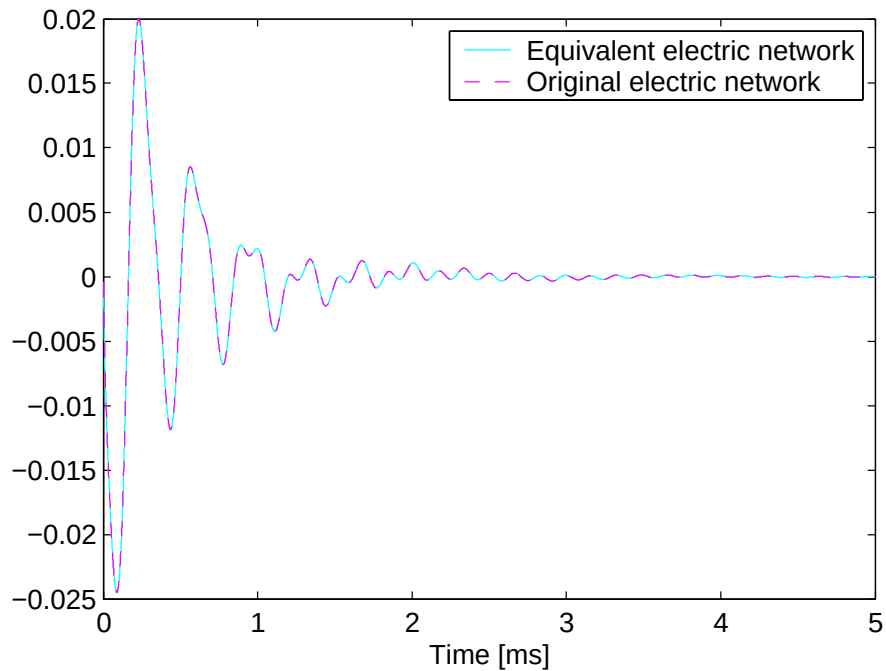


Fig. 8 Simulated current flowing into terminal 2

Note:

In this example is E in (5.3) virtually zero. The step response in fig. 8 can then also be calculated analytically as follows:

```

I=ones(Nc*N,1);
t=(0:1e-5:5e-3); Nt=length(t);
for k=1:Nt
    y=C*diag( A.^(-1).*(exp(A.*t(k))-I) ) *B +D;
    yy(k)=y(2,1);
end
figure(4),plot(1000*t,yy);
xlabel('Time [ms]'); ylabel('Current [A]');

```

7. REFERENCES

- [1] B. Gustavsen and A. Semlyen, "Rational approximation of frequency domain responses by Vector Fitting", *IEEE Trans. Power Delivery*, vol. 14, no. 3, July 1999, pp. 1052-1061.
- [2] B. Gustavsen and A. Semlyen, "Enforcing Passivity for Admittance Matrices Approximated by Rational Functions", *IEEE Trans. Power Systems*, pp. 97-104, vol. 16, no. 1, Feb. 2001.
- [3] B. Gustavsen, "Computer code for rational approximation of frequency dependent admittance matrices", *IEEE Trans. Power Delivery*, vol. 17, no. 4, Oct. 2002, pp. 1093-1098.